

ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA EN SISTEMAS



VERIFICACIÓN Y VALIDACIÓN DE SOFTWARE (ISWD752)

GR1SW

PROYECTO FINAL

**Análisis de Seguridad OWASP Top 10 (2021) — Hospital Management
System**

SERGIO RODRÍGUEZ

ISAAC PROAÑO

Fecha de presentación: 23 – 07 – 2026



Análisis Estático de Código — Hospital Management System

Código:

(ISWD752) GR1SW

Versión: 1

Elaborado: 23 – 7 – 2026

Página 2 de 16

Este informe consolida los hallazgos de seguridad detectados a lo largo de todo el proyecto (pruebas unitarias, integración, E2E y análisis estático — ver informes/hallazgos/), mapeados formalmente al estándar OWASP Top 10 (2021). Cada uno fue verificado empíricamente (no son solo lectura de código): con curl, tests de integración con MockMvc+H2, o Playwright contra el stack real.

1. Mapeo de vulnerabilidades al OWASP Top 10 (2021)

#	Vulnerabilidad	Categoría OWASP	Riesgo
1	Inyección SQL en búsqueda de doctores por especialidad	A03:2021 – Injection	Alto
2	XSS almacenado en diagnóstico de historias clínicas	A03:2021 – Injection	Alto
3	XSS DOM-based en showAlert() (todas las alertas del frontend)	A03:2021 – Injection	Alto
4	Ausencia total de autenticación y autorización	A01:2021 – Broken Access Control	Alto
5	CORS configurado con origen comodín (*) en los 4 controllers	A05:2021 – Security Misconfiguration	Medio
6	Fuga de información: stack traces y mensajes internos en respuestas de error	A05:2021 – Security	Medio



Análisis Estático de Código — Hospital Management System

Código:

(ISWD752) GR1SW

Versión: 1

Elaborado: 23 – 7 – 2026

Página 3 de 16

#	Vulnerabilidad	Categoría OWASP	Riesgo
		Misconfigurati on	
7	Credenciales de base de datos hardcodeadas en texto plano	A02:2021 – Cryptographic Failures	Medio
8	Falta de FK citas.paciente_id → citas huérfanas / inconsistencia de datos	A08:2021 – Software and Data Integrity Failures	Medio
9	Semántica HTTP incorrecta: ResourceNotFoundException devuelve 200 en vez de 404	A04:2021 – Insecure Design	Bajo
10	Ausencia de logging de errores del servidor	A09:2021 – Security Logging and Monitoring Failures	Medio
11	apiFetch ignora response.ok: estados de éxito/error falsos en el cliente	A08:2021 – Software and Data Integrity Failures	Alto (impac to operativo)



Análisis Estático de Código — Hospital Management System

Código:

(ISWD752) GR1SW

Versión: 1

Elaborado: 23 – 7 – 2026

Página 4 de 16

11 vulnerabilidades documentadas, todas con evidencia reproducible.

2. Detalle de cada hallazgo

1. Inyección SQL — DoctorService.buscarPorEspecialidadInsegura

- **Ubicación:** backend/src/main/java/com/hospital/service/DoctorService.java:57-61, expuesto
vía backend/src/main/java/com/hospital/controller/DoctorController.java:50-53 (GET /api/doctores/buscar-especialidad?q=...)
- **Categoría OWASP:** A03:2021 – Injection
- **Riesgo:** Alto
- **Impacto potencial:** Un atacante puede leer, filtrar o (con payloads más elaborados, dependiendo de permisos de la cuenta de BD) modificar/eliminar datos de cualquier tabla accesible por el usuario admin de PostgreSQL, ya que la query se construye por concatenación directa ("...ILIKE %" + especialidad + "%") sin parametrizar.
- **Evidencia:**
 - curl -G "http://localhost:8080/api/doctores/buscar-especialidad" \
 - --data-urlencode "q=' OR '1'='1' -- "
 - # Devuelve los 4 doctores (bypass total del filtro) en vez de 0

Confirmado además operando la búsqueda desde la UI real (Playwright, frontend/e2e/doctores.spec.js):



Análisis Estático de Código — Hospital Management System

Código:
(ISWD752) GR1SW

Versión: 1

Elaborado: 23 – 7 – 2026

Página 5 de 16

Hospital Management System

Dashboard Pacientes Doctores Citas Historias Clínicas

Doctores

'OR '1'='1' -- + Nuevo Doctor

NOMBRE COMPLETO	ESPECIALIDAD	EMAIL	CONSULTORIO	ACCIONES
Elena Rodriguez	Cardiologia	elena.rodriguez@hospital.com	CONS-101	<button>Editar</button> <button>Eliminar</button>
Miguel Torres	Pediatrica	miguel.torres@hospital.com	CONS-205	<button>Editar</button> <button>Eliminar</button>
Sofia Castillo	Dermatologia	sofia.castillo@hospital.com	CONS-304	<button>Editar</button> <button>Eliminar</button>
Diego Morales	Sin especialidad	diego.morales@hospital.com	CONS-150	<button>Editar</button> <button>Eliminar</button>

Hospital Management System — Proyecto de Validación y Verificación de Software — EPN 2026A

- **Nota de herramientas:** SpotBugs+FindSecBugs **no detectó** esta vulnerabilidad automáticamente (ver informes/analisis-estatico.md, hallazgo 1) por limitaciones de taint-tracking con la concatenación de strings vía invokedynamic en JDK 17+. Confirma que el análisis estático no reemplaza la revisión manual ni las pruebas de penetración.
- **Mitigación:** usar @Query parametrizada (JPQL o SQL nativo con :parametro) o el método ya existente y seguro DoctorRepository.findByEspecialidadContainingIgnoreCase. Eliminar el método inseguro o restringir su uso.
- **Verificación:** ejecutar el curl de arriba; si la respuesta contiene más registros de los que coinciden literalmente con el payload como texto de especialidad, la vulnerabilidad está presente.

2. XSS almacenado — diagnóstico de Historia Clínica

- **Ubicación:** Backend: HistoriaClinicaService.crear() (backend/src/main/java/com/hospital/service/HistoriaClinicaService.java:40-59, sin sanitizar diagnostico). Frontend: frontend/js/historias.js:53-74 (renderTabla,



Análisis Estático de Código — Hospital Management System

Código:

(ISWD752) GR1SW

Versión: 1

Elaborado: 23 – 7 – 2026

Página 6 de 16

inserta `{h.diagnostico}` vía `innerHTML` sin escapar) y `historias.js:135-168` (`verHistoria`, mismo patrón).

- **Categoría OWASP:** A03:2021 – Injection (Cross-Site Scripting)
- **Riesgo:** Alto
- **Impacto potencial:** Un usuario con acceso al formulario de historias clínicas puede almacenar JavaScript que se ejecutaría en el navegador de **cualquier otro usuario** que visualice esa historia (robo de sesión, phishing interno, pivoteo dentro de la red del hospital).
- **Evidencia:** confirmado a nivel de API en `HistoriaClinicaControllerIntegrationTest` (Paso 3): el backend persiste y devuelve `<script>...</script>` tal cual. Confirmado también con Playwright (`frontend/e2e/historias.spec.js`), ingresando el payload directamente en el formulario real:

The screenshot displays the 'Formulario de Historia Clínica' (Clinical History Form) modal in the Hospital Management System. The modal is overlaid on a blurred background of the system's main interface. The form contains the following fields:

- Paciente ***: A dropdown menu with 'Juan Perez' selected.
- Doctor**: A dropdown menu with 'Elena Rodriguez' selected.
- Diagnóstico ***: A text area containing the malicious payload: `<img src=x onerror="window.__xssEjecutado = true">`.
- Tratamiento**: A text area with the value 'Tratamiento de prueba E2E'.
- Observaciones**: An empty text area.

At the bottom of the modal are two buttons: 'Cancelar' (Cancel) and 'Guardar' (Save). The background interface shows a navigation bar with 'Dashboard', 'Pacientes', 'Doctores', 'Citas', and 'Historias Clínicas'. The 'Historias Clínicas' section is active, showing a sidebar with 'PACIENTE' and 'DOCTOR' tabs, and a main area with a '+ Nueva Historia Clínica' button and an 'ACCIONES' section.

- **Hallazgo adicional interesante:** actualmente **no es explotable vía la UI real** porque el listado de historias está roto por un bug no relacionado (BUG-01, serialización de proxies LAZY de Hibernate — ver informes/hallazgos/03 y 08), así que no existen



Análisis Estático de Código — Hospital Management System

Código:

(ISWD752) GR1SW

Versión: 1

Elaborado: 23 – 7 – 2026

Página 7 de 16

filas ni botón "Ver" que disparen el renderizado. Esto no significa que la vulnerabilidad esté mitigada: sigue almacenada en la base de datos y sería explotable en cuanto se corrija el bug de serialización (o si otro cliente/integración consume la API directamente).

- **Mitigación:** sanitizar el HTML en el backend antes de persistir (p. ej. con OWASP Java HTML Sanitizer) y/o escapar el contenido en el frontend al insertarlo en el DOM (usar `textContent` en vez de `innerHTML`, o pasar por una función de escape completa).
- **Verificación:** crear una historia clínica con `diagnostico = "<img src=x onerror=alert(1)>"` vía POST `/api/historias-clinicas` y comprobar que el GET posterior devuelve el payload sin escapar.

3. XSS DOM-based — `showAlert()` en todas las notificaciones del frontend

- **Ubicación:** `frontend/js/utils.js:66-77`
- **Categoría OWASP:** A03:2021 – Injection (XSS)
- **Riesgo:** Alto
- **Impacto potencial:** `showAlert(message, type)` inserta `message` directamente vía `container.innerHTML` sin escapar. Cualquier llamada a `showAlert()` con datos controlados por el usuario (mensajes de error que incluyan valores ingresados, por ejemplo) es un vector de XSS. Confirmado en `frontend/js/__tests__/utils.test.js` con el payload `<img src=x onerror="window.__xss = true">`, insertado y verificado como elemento real del DOM.
- **Evidencia:** test unitario Jest (`utils.test.js`, describe `showAlert`) y lectura directa del código: `container.innerHTML = \`

`\${message}`

``;`



Análisis Estático de Código — Hospital Management System

Código:

(ISWD752) GR1SW

Versión: 1

Elaborado: 23 – 7 – 2026

Página 8 de 16

- **Mitigación:** usar `textContent` para el mensaje, o pasar `message` por `escapeHTML()` antes de interpolarlo (y además corregir `escapeHTML` — ver Hallazgo 3 de informes/hallazgos/04-paso4-unitarias-frontend.md, no escapa comillas simples ni backticks).
- **Verificación:** invocar `showAlert('<img src=x onerror=alert(1)>')` desde la consola del navegador con la app cargada; el `<img>` se inserta como elemento real.

4. Ausencia total de autenticación y autorización

- **Ubicación:** todo `backend/src/main/java/com/hospital/controller/*.java`; **no** existe dependencia `spring-boot-starter-security` en `backend/pom.xml` (verificado: `grep -n Security pom.xml` no devuelve resultados)
- **Categoría OWASP:** A01:2021 – Broken Access Control (también relacionable con A07:2021 – Identification and Authentication Failures)
- **Riesgo:** Alto
- **Impacto potencial:** cualquier persona con acceso de red al backend puede leer, crear, modificar o eliminar **todos los datos** (pacientes, doctores, citas, historias clínicas — información médica sensible) sin ningún tipo de credencial. No hay distinción entre "administrador", "doctor" o "paciente" como lo sugieren las historias de usuario del propio proyecto (HU-01 a HU-04 mencionan roles).
- **Evidencia:** `curl http://localhost:8080/api/pacientes` (sin ningún header de autenticación) devuelve los datos completos, incluyendo DELETE, POST, PUT sin restricción, como se usó libremente en todos los tests de este proyecto.
- **Mitigación:** implementar Spring Security con autenticación (JWT o sesión) y autorización basada en roles (`@PreAuthorize`) acorde a los roles mencionados en las historias de usuario (administrador, doctor).



Análisis Estático de Código — Hospital Management System

Código:

(ISWD752) GR1SW

Versión: 1

Elaborado: 23 – 7 – 2026

Página 9 de 16

- **Verificación:** cualquier petición a cualquier endpoint sin header Authorization responde con datos/éxito en vez de 401/403.

5. CORS con origen comodín (*)

- **Ubicación:** PacienteController.java:14, DoctorController.java:14, CitaController.java:15, HistoriaClinicaController.java:14 — todos con `@CrossOrigin(origins = "*")`
- **Categoría OWASP:** A05:2021 – Security Misconfiguration
- **Riesgo:** Medio (se agrava por la falta de autenticación del Hallazgo 4: sin login, el impacto de un CORS abierto es menor que en un sistema con sesión/cookies, pero sigue siendo mala práctica y se vuelve crítico en cuanto se agregue autenticación basada en cookies)
- **Impacto potencial:** cualquier sitio web, ejecutado en el navegador de cualquier usuario que visite ese sitio, puede hacer peticiones al backend del hospital y leer las respuestas. Combinado con el Hallazgo 4 (sin auth), esto no agrega superficie de ataque adicional *hoy*, pero es una configuración peligrosa que quedaría explotable de inmediato si se agrega autenticación por cookies sin revisar este punto.
- **Evidencia:** literal en el código, comentado incluso como // BUG INTENCIONAL: CORS demasiado permisivo (OWASP) en PacienteController.java:14.
- **Mitigación:** restringir origins a la lista real de dominios del frontend (p. ej. `http://localhost:3000` en desarrollo, dominio real en producción) vía configuración centralizada (`WebMvcConfigurer` o `application.properties`), no hardcodeado por controller.
- **Verificación:** inspeccionar el header `Access-Control-Allow-Origin: *` en la respuesta de cualquier endpoint.

6. Fuga de información en respuestas de error (stack traces y mensajes internos)

- **Ubicación:** `backend/src/main/java/com/hospital/exception/GlobalExceptionHandler.java:28-57`



Análisis Estático de Código — Hospital Management System

Código:

(ISWD752) GR1SW

Versión: 1

Elaborado: 23 – 7 – 2026

Página 10 de 16

- **Categoría OWASP:** A05:2021 – Security Misconfiguration
- **Riesgo:** Medio
- **Impacto potencial:** todo error 500 devuelve el stack trace completo (ex.getStackTrace()) en el body JSON, revelando rutas de archivo del servidor, versiones de librerías, nombres de clases internas y estructura del código — información valiosa para reconocimiento previo a un ataque. Los errores 400 exponen además el mensaje interno completo de la excepción de validación de Spring en el campo debug.
- **Evidencia:** curl http://localhost:8080/api/citas (dispara el bug de serialización, BUG-01) devuelve un JSON con "stackTrace":[{...100+ frames...}].
- **Mitigación:** en GlobalExceptionHandler, loguear la excepción completa server-side (logger.error(...), resolviendo también el Hallazgo 10) y devolver al cliente solo un identificador de error genérico y un mensaje seguro.
- **Verificación:** provocar cualquier 500 (p. ej. GET /api/citas) y confirmar que el body contiene stackTrace.

7. Credenciales de base de datos hardcodeadas en texto plano

- **Ubicación:** backend/src/main/resources/application.properties:7-8 (spring.datasource.password=hospital123) y docker-compose.yml:11 (POSTGRES_PASSWORD: hospital123)
- **Categoría OWASP:** A02:2021 – Cryptographic Failures
- **Riesgo:** Medio (se agrava si el repositorio es público, ya que la contraseña queda expuesta en el historial de Git indefinidamente)
- **Impacto potencial:** cualquiera con acceso al código fuente (incluyendo el propio repositorio de este proyecto académico) conoce la contraseña de administrador de la base de datos.



Análisis Estático de Código — Hospital Management System

Código:

(ISWD752) GR1SW

Versión: 1

Elaborado: 23 – 7 – 2026

Página 11 de 16

- **Evidencia:** valores en texto plano, idénticos en ambos archivos.
- **Mitigación:** usar variables de entorno (`{DB_PASSWORD}`) inyectadas en tiempo de despliegue, o un gestor de secretos (Vault, AWS Secrets Manager, etc.), nunca committeadas al repositorio.
- **Verificación:** `grep -rn "hospital123" .` en el repositorio.

8. Falta de FOREIGN KEY citas.paciente_id → integridad de datos

- **Ubicación:** backend/src/main/resources/schema.sql:37-46 (comentario explícito: -- BUG: falta FOREIGN KEY REFERENCES pacientes(id)), backend/src/main/java/com/hospital/model/Cita.java:14-17
- **Categoría OWASP:** A08:2021 – Software and Data Integrity Failures
- **Riesgo:** Medio
- **Impacto potencial:** se pueden crear citas referenciando pacientes inexistentes (confirmado en CitaServiceTest y CitaControllerIntegrationTest, Pasos 2 y 3: POST /api/citas con pacienteId: 999999 responde 200 sin error). Esto puede generar reportes médicos inconsistentes o errores en cascada en otros módulos que asuman que toda cita tiene un paciente válido.
- **Evidencia:** CitaControllerIntegrationTest.crear_conPacienteIdInexistente_seCreaSinValidar (Paso 3).
- **Mitigación:** agregar CONSTRAINT fk_citas_paciente FOREIGN KEY (paciente_id) REFERENCES pacientes(id) en schema.sql y cambiar el mapeo de Cita.pacienteId a una relación @ManyToOne real (como ya tiene doctor), y validar explícitamente en CitaService.crear().
- **Verificación:** POST /api/citas con un pacienteId que no exista en la tabla pacientes; debería rechazarse y actualmente no lo hace.

9. Semántica HTTP incorrecta — 200 en vez de 404



Análisis Estático de Código — Hospital Management System

Código:

(ISWD752) GR1SW

Versión: 1

Elaborado: 23 – 7 – 2026

Página 12 de 16

- **Ubicación:** GlobalExceptionHandler.java:17-26
- **Categoría OWASP:** A04:2021 – Insecure Design
- **Riesgo:** Bajo
- **Impacto potencial:** clientes automatizados (incluyendo el propio frontend) que verifiquen el código de estado HTTP para decidir el flujo (p. ej. manejo de caché, reintentos, o un futuro API gateway) tratarán un "no encontrado" como éxito, pudiendo enmascarar errores reales en integraciones futuras.
- **Evidencia:** PacienteControllerIntegrationTest.buscar_conIdInexistente_documenta BugDeStatus200 (Paso 3): GET /api/pacientes/9999 responde 200 OK con {"status":404, ...} en el body.
- **Mitigación:** return ResponseEntity.status(HttpStatus.NOT_FOUND).body(body); en vez de ResponseEntity.ok(body).
- **Verificación:** curl -i http://localhost:8080/api/pacientes/99999 — el código de estado HTTP real es 200.

10. Ausencia de logging de errores del servidor

- **Ubicación:** GlobalExceptionHandler.java:46-57 (handleGeneral)
- **Categoría OWASP:** A09:2021 – Security Logging and Monitoring Failures
- **Riesgo:** Medio
- **Impacto potencial:** ninguna excepción no controlada queda registrada en los logs del servidor (confirmado: ninguna línea ERROR aparece en consola pese a decenas de 500 provocados durante todo el proyecto). En producción, esto imposibilita detectar patrones de ataque (múltiples intentos de inyección, escaneo de endpoints) o diagnosticar incidentes reales sin depender exclusivamente de que el cliente reporte el stack trace que se le filtró (Hallazgo 6).



Análisis Estático de Código — Hospital Management System

Código:

(ISWD752) GR1SW

Versión: 1

Elaborado: 23 – 7 – 2026

Página 13 de 16

- **Evidencia:** logs de consola capturados en Pasos 1 y 3 — ninguna traza de las excepciones 500 generadas.
- **Mitigación:** agregar `private static final Logger log = LoggerFactory.getLogger(GlobalExceptionHandler.class);` y `log.error("Error no controlado", ex);` en `handleGeneral`, e integrar una herramienta de monitoreo (ELK, Sentry, etc.) en un entorno real.
- **Verificación:** provocar un 500 y confirmar la ausencia de cualquier línea de log correspondiente en la consola del backend.

11. apiFetch ignora response.ok: estados de éxito/error falsos

- **Ubicación:** `frontend/js/api.js:19-39`
- **Categoría OWASP:** A08:2021 – Software and Data Integrity Failures (integridad de la información mostrada al usuario)
- **Riesgo:** Alto (impacto operativo, no técnico-explotable, pero crítico en un sistema hospitalario)
- **Impacto potencial:** confirmado con Playwright en el Paso 5 con dos variantes opuestas, ambas graves:
 - Eliminar un doctor **con** citas asociadas: el backend rechaza el borrado (500), pero el usuario ve **"Doctor eliminado exitosamente"** — un falso positivo que podría hacer creer al personal administrativo que un doctor fue dado de baja cuando sigue activo.
 - Eliminar un paciente/doctor **sin** dependencias: el borrado sí se ejecuta (200), pero el usuario ve **"Error al eliminar..."** — un falso negativo que podría llevar a reintentos innecesarios o pérdida de confianza en el sistema.
- **Evidencia:** `frontend/e2e/doctores.spec.js` (test "bug crítico"), `frontend/e2e/pacientes.spec.js`. Detalle completo en `informes/hallazgos/06-paso5-e2e-doctores.md`.



Análisis Estático de Código — Hospital Management System

Código:
(ISWD752) GR1SW

Versión: 1

Elaborado: 23 – 7 – 2026

Página 14 de 16

Hospital Management System

Dashboard Pacientes Doctores Citas Historias Clínicas

Doctor eliminado exitosamente

Doctores

Buscar por especialidad... + Nuevo Doctor

NOMBRE COMPLETO	ESPECIALIDAD	EMAIL	CONSULTORIO	ACCIONES
Elena Rodriguez	Cardiologia	elena.rodriguez@hospital.com	CONS-101	Editar Eliminar
Miguel Torres	Pediatrica	miguel.torres@hospital.com	CONS-205	Editar Eliminar
Sofia Castillo	Dermatologia	sofia.castillo@hospital.com	CONS-304	Editar Eliminar
Diego Morales	Sin especialidad	diego.morales@hospital.com	CONS-150	Editar Eliminar

Hospital Management System — Proyecto de Validación y Verificación de Software — EPN 2026A

Hospital Management System

Dashboard Pacientes Doctores Citas Historias Clínicas

Error al eliminar paciente

Pacientes

Buscar por nombre... + Nuevo Paciente

NOMBRE COMPLETO	EMAIL	TELÉFONO	FECHA NACIMIENTO	ESTADO	ACCIONES
Juan Perez	juan.perez@email.com	0991234567	14 de marzo de 1985	Activo	Ver Editar Eliminar
Maria Garcia	maria.garcia@email.com	0987654321	21 de julio de 1990	Activo	Ver Editar Eliminar
Carlos Lopez	carlos.lopez@email.com	0976543210	29 de noviembre de 1978	Activo	Ver Editar Eliminar
Ana Martinez	ana.martinez@email.com	0965432109	9 de enero de 2000	Activo	Ver Editar Eliminar
Pedro Ramirez	pedro.ramirez@email.com	0954321098	4 de junio de 1965	Inactivo	Ver Editar Eliminar
E2E1784857843654 Paciente	E2E1784857843654@e2e-test.com	0987654321	31 de diciembre de 1969	Activo	Ver Editar Eliminar

Hospital Management System — Proyecto de Validación y Verificación de Software — EPN 2026A

- **Mitigación:** en apiFetch, verificar response.ok y lanzar una excepción con la información del error si es false, permitiendo que el código que llama distinga correctamente éxito de fallo.
- **Verificación:** eliminar (vía UI) un doctor con citas asociadas y observar el mensaje mostrado frente al estado real de los datos tras recargar.



Análisis Estático de Código — Hospital Management System

Código:

(ISWD752) GR1SW

Versión: 1

Elaborado: 23 – 7 – 2026

Página 15 de 16

3. Resumen de riesgo

Riesgo	Cantidad
Alto	5
Medio	5
Bajo	1

4. Recomendaciones generales (priorizadas)

1. **Implementar autenticación/autorización** (Spring Security) — sin esto, el resto de mitigaciones son secundarias.
2. **Parametrizar la query de buscarPorEspecialidadInsegura** — la inyección SQL es la vulnerabilidad más crítica y de más fácil explotación.
3. **Sanitizar/escapar toda salida hacia el DOM** (backend y frontend) — resuelve los dos XSS de un solo esfuerzo si se centraliza en una función de sanitización compartida.
4. **Revisar apiFetch para respetar response.ok** — bajo costo de implementación, alto impacto en confiabilidad percibida por el usuario final.
5. Ajustar CORS, logging de errores, y ocultar detalles internos en respuestas — mejoras de "higiene" de configuración, rápidas de aplicar.
6. Agregar la FK faltante y corregir la semántica HTTP — deuda técnica de diseño, útil documentarla aunque no se corrija en este ciclo.

5. Referencias cruzadas



Análisis Estático de Código — Hospital Management System

Código:

(ISWD752) GR1SW

Versión: 1

Elaborado: 23 – 7 – 2026

Página **16** de **16**

Este informe se apoya en evidencia generada durante todo el proyecto:

- informes/hallazgos/00-entorno-y-bugs-iniciales.md — bugs intencionales confirmados en el código base
- informes/hallazgos/03-paso3-integracion-backend.md — PoC de SQL injection, causa raíz de BUG-01, violación de FK
- informes/hallazgos/04-paso4-unitarias-frontend.md — XSS DOM en showAlert, comportamiento real de apiFetch
- informes/hallazgos/05 a 08-paso5-e2e-*.md — evidencia end-to-end con capturas de pantalla
- informes/analisis-estatico.md — hallazgos de Checkstyle/SpotBugs/FindSecBugs/ESLint